

8 FUNCTIONS

You may be familiar with the mathematical concept of a function. A function is a relationship or mapping between one or more inputs and a set of outputs. In mathematics, a function is typically represented like this:

$$z = f(x, y)$$

Here f is a function that operates on the inputs x and y . The output of the function is z . However, programming functions are much more generalized and versatile than this mathematical definition. In fact, appropriate function definition and use is so critical to proper software development that virtually all modern programming languages support both built-in and user-defined functions.

In programming, a function is a self-contained block of code that encapsulates a specific task or related group of tasks. Previously we have been introduced to some of the built-in functions provided by Python such as `len()`, `id()`. Both these functions take one argument and `len()` returns the length of the object and `id()` returns the object's unique integer identifier.

```
>>> msg = "Hello, All!"
>>> numbers = list(range(1,10))
>>> print(msg, 'contains', len(msg), 'characters')
Hello, All! contains 11 characters
>>> print(numbers, 'contains', len(numbers), 'numbers')
[1, 2, 3, 4, 5, 6, 7, 8, 9] contains 9 numbers
```

The `len()` function returns the number of items in a given sequence. At first, we have passed `msg` as input to `len()`, returns the number of characters in “Hello, All!”. Later a list is passed as an argument and it returns the number of items in the list `numbers` and `id()` returns the unique identification number of the specified object.

```
>>> id(msg)
1815290128560
>>> id(numbers)
1815290131776
```

Each of these built-in functions performs a specific task. The code that performs the specified task is defined somewhere, but we don't need to think about the working of the code. Only we need to know about the interface of the function which may or may not have the following: (a) one or many arguments, it takes as input and (b) one or many values it returns.

In the examples you have seen that, we have passed the appropriate arguments to `id()` and `len()`. After the successful execution of a function call both functions return a value. Once we call a function, program execution goes off to the designated body of the code and does its useful thing. When the function body is finished, program execution returns to the code from where it was left off. Let see the pictorial view of a function call:

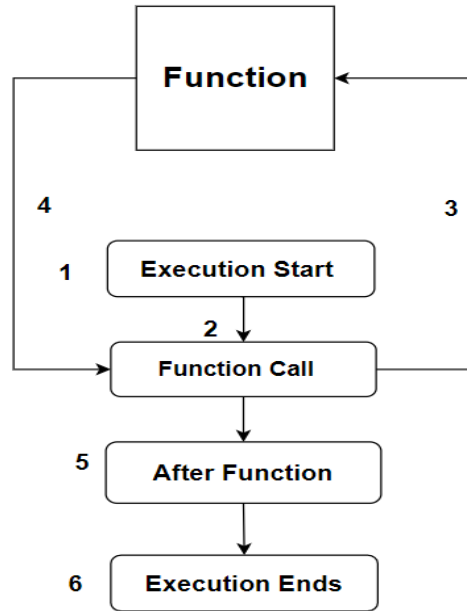


Figure 8.1: Flow of a program execution with a function call

In Fig. 8.1, we have depicted a flow of program execution and shown in a sequence of numbers from 1 – 6. The box written with Function caption represents a definition of the function and program execution starts from sequence number 1. At this point your script file can have any valid python statements, such as declaration or assignment. Thereafter at sequence number 2 the function calls get executed. Due to this, the program control has jumped (mentioned as number 3) inside the function, to execute the line of statements.

After the successful execution of the function, program control returns to the same point (represented as sequence number 2) of the script, from where the control was jumped. While the program control is returning, it can have some return value as well. Thereafter, program execution will continue till the execution of the last line of code. When you define your own Python function, it works just the same. From somewhere in your code, you'll call your Python function and program execution will transfer to the body of the code that makes up the function.

When the function is finished, execution returns to the location where the function was called. Depending on how you designed the function's interface, data may be passed in when the function is got called and return values may be passed back when it finishes.

8.1 Importance of Python Function

Virtually all programming languages support a form of user-defined functions, although they aren't always called functions. In other languages, you may see them referred to as, (a) subroutines; (b)

procedures; (c) methods; and (d) subprograms. So, why bother defining functions? There are several very good reasons. Let's go over a few now.

8.1.1 Abstraction and Reusability

Suppose you write some code that does something useful. As you continue development, you find that the task performed by that code is one you need often, in many different locations within your application. What should you do? Well, you could just replicate the code repeatedly, using your editor's copy-and-paste capability.

Remember it looks like a reasonable solution, but later the maintenance of the code will be very difficult. Later you decided that the code requires some modification. You'll either find something wrong with it that needs to be fixed, or you'll want to enhance it in some way. If copies of the code are scattered all over your application, then you'll need to make the necessary changes in every location.

So, the solution to the above problem is to define a Python function that performs the task. Anywhere in your application that you need to accomplish the task, you simply call the function. Down the line, if you decide to change how it works, then you only need to change the code in one location (the box written with Function caption in Figure 8.1), which is the place where the function is defined. The changes will automatically be picked up anywhere the function is called. The abstraction of functionality into a function definition is an example of the "Don't Repeat Yourself (DRY) Principle" of software development. This is arguably the strongest motivation for using functions.

8.1.2 Modularity

Functions allow complex processes to be broken up into smaller steps. Imagine, for example, that you have a program that reads in a file, processes the file contents, and then writes an output file. Your code could be look like this:

```

1  # Main Program
2  # Opening a file for reading
3  statements1
4  statements2
5  statements3
6
7  # Processing file content
8  statements1
9  statements2
10 statements3
11
12 # Updating file
13 statements1
14 statements2
15 statements3

```

Figure 8.2: Putting Everything in a Main Program

In Fig. 8.2, we have listed three different segments under the main program and have put all the code segments together in a long sequence. Further, if we consider the whitespace and comments to help

organize the code, the length of the script will increase. However, if the code were to get much lengthier, it would become very difficult to manage the code. Instead of this we can use the modular approach to separate the code segments using functions and later we can call the functions accordingly. A simple modular approach of python script is shown in Fig. 8.3.

```

1  # Opening a file for reading
2  ~def fileRead():
3      statements1
4      statements2
5  # Processing file content
6  ~def fileProcess():
7      statements1
8      statements2
9  # Updating file
10 ~def fileWrite():
11     statements1
12     statements2
13
14 # Main Program
15 fileRead()
16 fileProcess()
17 fileWrite()

```

Figure 8.3: A Sample of Modular approach using Python function

Instead of all the code being put together, it's broken out into separate functions, each of which focuses on a specific task. Those tasks are reading, processing, and writing. The main program now simply needs to call each of these in turn. Breaking a large task into smaller, bite-sized sub-tasks helps make the large task easier to think about and manage. As programs become more complicated, it becomes increasingly beneficial to modularize them in this way.

8.1.3 Namespace Separation

A namespace is a region of a program in which identifiers have meaning. As you'll see below, when a Python function is called, a new namespace is created for that function, one that is distinct from all other namespaces that already exist. The practical upshot of this is that variables can be defined and used within a Python function even if they have the same name as variables defined in other functions or in the main program. In these cases, there will be no confusion or interference because they're kept in separate namespaces.

This means that when you write code within a function, you can use variable names and identifiers without worrying about whether they're already used elsewhere outside the function. This helps minimize errors in code considerably. Till this point of study, we have tried to make you understand these two questions: (a) What function can have or not? (b) Why are python functions important?

In the next part of this Chapter, we will introduce various techniques related to Python function, though we can achieve better code reusability and maintenance of the code will be easy.

8.2 Define and call a function

A usual syntax to defining a function in Python is given below and each component are discussed in Table 8.1:

```
>>> def function_name(parameters):
...     statement(s)
```

Table 8.1: Details of each component of a Python function

Component	Meaning
Def	The keyword that informs Python that a function is being defined
function_name	A valid python identifier name
parameters	An optional, comma-separated list of parameters that may be passed to the function
:	Punctuation that denotes the end of the Python function header (the name and parameter list)
statement(s)	A block of valid Python statements

The **statement(s)**, is called the body of the function. The body is a block of statements that will be executed when the function is called. The body of a function is defined by indentation just like other control structures such as: an **if** or **while** statement. Once the function definition is completed, we need to call the function so that code associated within the function gets executed. Consider the syntax given below to call a function:

```
>>> function_name(arguments)
```

arguments are the values passed into the function. They correspond to the **parameters** in the function definition. You can define a function that doesn't take any arguments, but the parentheses are still required after a function name either in definition or a function call, even if they're empty. Let us start with a basic example to understand the basic concept of python function:

```
1 def fun():
2     msg = 'inside fun'
3     print(msg)
4
5 print('Start of Program')
6 fun()
7 print('End of Program')
```

```
Start of Program
inside fun
End of Program
```

Details of the above sample code are discussed below:

- Line 1: uses the **def** keyword to indicate that a function is being defined. Execution of the **def** statement creates the definition for the function **fun()**. All the following lines that are indented (lines 2 to 3) become part of the body of **fun()** and are stored as its definition, but they aren't executed yet.

- Line 4: a whitespace between the function definition and the first line of the main program just improves the code readability. This whitespace segregates the function definition from the main script.
- Line 5: is the first statement that isn't indented because it isn't a part of the definition of `fun()`. It's the start of the main program and when the executes, this statement is executed first.
- Line 6: is a call to `fun()`. Note that empty parentheses are always required in both a function definition and a function call, even when there are no parameters or arguments. Execution proceeds to `fun()` and then statements in the body of `fun()` are executed. A sequence of program execution is depicted in Figure 8.1.
- Line 7: is the next line to execute once the body of `fun()` has finished. Execution returns to this `print()` statement.

8.2.1 Defining an empty function

Sometimes, you may want to define an empty function that does nothing. This is referred to as a stub, which is usually a temporary placeholder for a Python function that will be fully implemented later. Just as a block in a control structure can't be empty, neither can the body of a function. To define a stub function, use the `pass` statement, as follows:

```
1 def fun():
2     pass
3
4 print('Start of Program')
5 fun()
6 print('End of Program')
```

```
Start of Program
End of Program
```

We can see that calling an empty function is syntactically correct but doesn't add any value to the output.

8.3 Types of Function Arguments

So far, we have discussed how to define a function and call a function but the functions we have defined haven't taken any arguments. That can sometimes be useful, and you'll occasionally write such functions. More often, though, you'll want to pass data into a function so that its behavior can vary from one invocation to the next. Let's see how to do that.

8.3.1 Positional Arguments

The most straightforward way to pass arguments to a Python function is with positional arguments (also called required arguments). In the function definition, you can specify a list of comma-separated parameters inside the parentheses, as shown below:

```
1 #defining a function with two parameters
2 def fun(name, age):
3     print(f'{name} is {age} years old')
4
5 #Calling the fun
```

```
6 fun('Tridib', 7)
7 fun('Goutam', 37)
```

Tridib is 7 years old
Goutam is 37 years old

A function `fun(name, age)` is defined with two parameters `name` and `age`. These parameters behave like variables that are defined locally to the function. When the function is called `fun('Tridib', 7)` at line 6, are bound to the parameters in order, as though by variable assignment and `'Rahul'` and `45` is assigned to `name` and `age` in order. While calling a function following points need to take care:

- The number of parameters in the function definition should match the numbers of arguments in the function call. Now let's try to call the above function with one argument name, as follows:

```
>>>fun('Rahul')
Traceback (most recent call last):
  File "<pyshell#8>", line 1, in <module>
    fun('Rahul')
TypeError: fun() missing 1 required positional argument: 'age'
```

Interpreter raised a `TypeError` as one positional argument `age` is missing. Even though, if we pass more number arguments than the required number, interpreter will raise a `TypeError`. See the function call given below with the error:

```
>>>fun('Rahul', 25, 'Male')
Traceback (most recent call last):
  File "<pyshell#0>", line 1, in <module>
    fun('Rahul', 25, 'Male')
TypeError: fun() takes 2 positional arguments but 3 were given
```

- We need to take care of the order of arguments while calling a function. The order should match the order of the parameters in the function definition. There's nothing to stop you from specifying positional arguments out of order, of course:

```
>>>fun(35, 'Rahul')
35 is Rahul years old
```

The function may even still run, but due to change of order in the arguments we did not get the desired output because 35 is assigned to `name` and Rahul is assigned to `age`. It's the responsibility of the programmer who defines the function to document what the appropriate arguments should be, and it's the responsibility of the user of the function to be aware of that information and abide by it.

Positional arguments are conceptually straightforward to use, but they're not very forgiving. You must specify the same number of arguments in the function call as there are parameters in the definition, and in the same order. Later in this chapter, you will find some argument-passing techniques that relax these restrictions.

8.3.2 Keyword Arguments

When you're calling a function, you can specify arguments in the form `keyword = value`. In this case, each `keyword` must match a parameter name present in the function definition. For example, the

previously defined function `fun()` may be called with keyword arguments as follows:

```
>>>fun(name = 'Rahul', age = 45)
```

```
Rahul is 45 years old
```

While we are calling the function with keyword order of parameter is not important anymore. Consider the next function call given below:

```
>>>fun(age = 45, name = 'Rahul')
```

```
Rahul is 45 years old
```

Now see the above function call, we didn't follow the order of the parameters. Using keyword arguments lifts the restriction on argument order. Each keyword argument explicitly designates a specific parameter by name, so you can specify them in any order and Python will still know which argument goes with which parameter. Referencing a keyword that doesn't match any of the declared parameters generates a **TypeError**. See the below function call:

```
>>>fun(nam = 'Rahul', age = 45)
Traceback (most recent call last):
  File "<pysHELL#10>", line 1, in <module>
    fun(nam = 'Rahul', age = 45)
TypeError: fun() got an unexpected keyword argument 'nam'
```

While calling the function we have put a wrong keyword name that does not match the function parameter. Instead of the given parameter **name**, we have used **nam** maybe knowingly or unknowingly and thus keywords didn't match during the function call and raised a **TypeError**. Like positional arguments, keyword arguments must match the number of parameters, or else raise a **TypeError**.

8.3.3 Default Parameters

If a parameter specified in a Python function definition has the form **name = value**, then **value** becomes a default value for that parameter. Parameters defined this way are referred to as default or optional parameters. An example of a function definition with default parameters is shown below:

```
1 def fun(a=3, b=4):
2     print(f'a = {a}, b = {b}')
```

In the function definition values 3 and 4 are assigned to the parameter 'a' and 'b'. During the function call, if we forget to pass value to any parameters, then the default value is considered. See the various form of function call `fun()`:

```
>>>fun()
>>>fun(10)
>>>fun(10, 20)
```

All these function calls are valid, and we will get the desired output. The first function call `fun()`, doesn't take any argument and the default values 3 and 4 is assumed for the parameters 'a' and 'b'. The second call `fun(10)`, has one argument, and values are assigned according to the position in

function definition from left to right. It means the default value for the parameter **'a'** is overwritten by 10 and default value for the parameter **'b'** will be retained. In the third call **fun(10,20)**, we have passed all the arguments and all the default values for the parameters are overwritten, means 10 and 20 is assigned to **'a'** and **'b'**. The output of all the function calls is given below:

```
a = 3, b = 4
a = 10, b = 4
a = 10, b = 20
```

The second call **fun(10)** is assigned value 10 to the parameter **a**, what if, we want to assign the passing argument to parameter **'b'** not **'a'**. To achieve this, we can mention the keyword at the time of function calling, provided all the parameters assigned with default value. See the function call given below:

```
>>>fun(b=9)
a = 3, b = 9
```

In the above function call default value for **'a'** is assumed and 9 is assigned to **'b'**. Now the question is can we mix default parameters with non-default parameters in a function definition. Let's us consider the function definitions given below:

```
def fun1(a, b = 4):
    print(f'a = {a}, b = {b}')

def fun1(a = 3, b):
    print(f'a = {a}, b = {b}')
```

In both the functions **fun1()**, **fun2()** definitions we have assigned only one default parameter. In **fun1()**, default argument follows non-default argument and in **fun2()**, non-default argument follows default argument. Now let's execute the above function and see if both the definitions are accepted by Python or not.

On execution you will get a syntax error, specifying that “non-default argument follows default argument” is pointed to the function definition **fun2()**. Means out of both the function definitions only **fun1()** is valid in Python. We must keep in mind that a function where default parameters are used must be assigned from right to left or for all the parameters. That's why definition for **fun1()** doesn't point to any SyntaxError.

This discussion will be incomplete if we don't use any mutable type as default parameter. Things can get weird if you specify a default parameter value that is a mutable object, specifically an empty list. Consider this Python function definition:

```
1 def fun(my_list = []):
2     my_list.append('e')
3     print(id(my_list), my_list)
4
5 fun()
6 fun(['a', 'b', 'c'])
7 fun()
```

First call **fun()**, takes a single list parameter, appends the string **e** to the end of the list, and prints the

id and the items of the list. Let's check the output:

```
1981250783552 ['e']
1981250785024 ['a', 'b', 'c', 'e']
1981250783552 ['e', 'e']
```

If we didn't pass any argument the default parameter is assumed, and an empty list is assigned to the parameter **my_list**. But between the consecutive function calls if we didn't pass any argument then the same list is referred. So, after the 3rd call, we are getting `['e', 'e']` as output, and the **id** of **my_list** is the same as first function call. While in the second call we have pass a list `['a', 'b', 'c']` as argument and in this case default value is overwritten and new **id** is referred.

In Python, default parameter values are defined only once when the function is defined (i.e., when the **def** statements are executed). The default value isn't re-defined each time the function is called. Thus, each time you call **fun()** without a parameter, you're performing **.append()** on the same list.

To stop these duplicate items due to the default list we can assign a default value **None** to the parameter. Later inside the function, we can check if the value of the parameter is **None** or not, and based on that we can create the list and then append the value. See the code solution given below:

```
1 def fun(my_list = None):
2     if my_list is None:
3         my_list = []
4     my_list.append('e')
5     print(id(my_list), my_list)
6
7 fun()
8 fun(['a', 'b', 'c'])
9 fun()
```

```
1471994294208 ['e']
1471994294208 ['a', 'b', 'c', 'e']
1471994294208 ['e']
```

In this way, we can ensure that an empty list will be created always if we didn't pass any argument while calling the function **fun()**.

8.4 Passing arguments by reference

In this section we have first highlighted two different approaches to passing an argument to a function. To discuss this, we have taken a reference from another programming language i.e., C and later in this Section, we have highlighted how Python passes the arguments to a function. In any programming language, there are two common ways to pass arguments to a function: (a) pass by value; and (b) pass by reference

In pass by value a copy of the argument is passed to a function, and changes made inside the function to the parameters aren't reflected outside of the function. Let us understand how arguments are passed by value to a function with a program written in C programming language depicted in Fig. 8.4.

```

1 #include<stdio.h>
2 void fun(int a){
3     a = 40;
4 }
5 int main(){
6     int a = 20;
7     fun(a);
8     printf("%d",a);
9 }

```

Figure 8.4: Sample C Program for passing arguments by Value

The function `void fun(int a)` (at line 2) is defined and get called inside another function `main()`. Like python, in C programming also, we need to call the function to execute the code written inside the function. In C programming, program execution started from the `main()` function (means every program written in C must have one `main()` function).

In the above sample code at first a variable '`a`' of type `int` is declared and initialized to 20 (at line 6). Thereafter '`a`' is passed as argument to the function `fun(a)` (at line 7). While calling the function the value of '`a`' is passed and assigned to the parameter '`a`' (listed in the function header at line 2) (a copy of argument '`a`'). Later inside the function, the `a`'s value is changed to 40 (at line 3) and program control is returned to the `main()` function.

In call by value, all the changes have been made to the copy of '`a`' which is created inside the `fun()`. So, the changes aren't reflected outside of the function `fun()` and the value of '`a`' remains unchanged. So, the `printf("%d", a)` statement prints the value of '`a`' as 20.

In call by reference, reference represents the address of a memory block. So instead of passing the value of the argument, we pass the address of the argument and changes made inside the function will be done over the actual argument and reflected outside of the function. In Fig. 8.5 the flow of call by value (in Fig. 8.5(a)) and call by reference (in Fig. 8.5(b)) is depicted.

While passing the argument in the call by reference we have passed the address of a `fun(&a)` and the changes made by the statement in the header these two arguments passing approaches and later we have re-written the above program and passed the argument by reference.

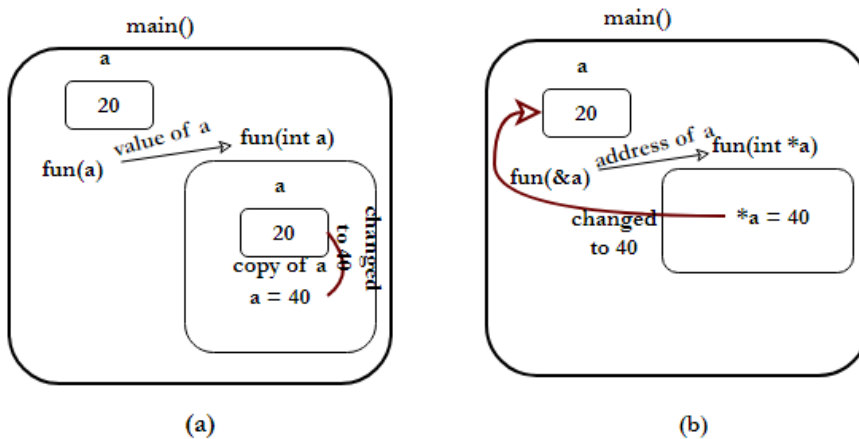


Figure 8.5: Passing argument (a) Call-by-value and (b) Call-by-reference

```

1 #include<stdio.h>
2 void fun(int *a){
3     *a = 40;
4 }
5 int main(){
6     int a = 20;
7     fun(&a);
8     printf("%d",a);
9 }

```

Figure 8.6: Sample C Program for passing arguments by Reference

See the above code at line 7 where the function `fun(int *a)` gets called and instead of the value of `'a'`, address of `(&a)` is passed. To hold the address of an argument, a pointer parameter type `int *a` is used in the function header at line 2. To access the memory block of `'a'`, instead of `a = 40`, `*a = 40` is used and the value `'a'` is changed to 40.

8.5 How does python pass the arguments?

In python arguments neither pass by value or reference exactly what it means in C programming. That's because in Python a reference doesn't mean quite the same thing as C programming. In Python, every piece of data is an object. So, a reference points to an object, not a specific memory location. That means the assignment isn't interpreted the same way in Python as it is in C. Consider the following statements in C:

```

int a = 20;
a = 40;

```

The first statement declares a variable `'a'` and refers to a memory block and later value 20 is kept on it. The next statement overwrites 20 and puts 40 on it. The scenario in python is quite different. Consider the statements given below:

```

>>>a = 20
>>>a = 40

```

The first statement causes `'a'` to point to an object whose value is 20. The next statement reassigns `'a'` to a new object whose value is 40. Stated another way, the second assignment rebinds `'a'` to a different object with value 40. In Python, when you pass an argument to a function, a similar rebinding occurs. Consider this example:

```

1 def fun(x):
2     x = 10
3
4 a = 5
5 fun(a)
6 print(a)

```

In this program, the statement `a = 5` (in line 4) creates a reference named `'a'` bound to an object whose value is 5. `fun()` is then called on line 5, with `'a'` as an argument. When the function `f()` executes first, a new reference called `'x'` is created, which initially points to the same 5 object as `'a'` does. However, when the statement `x = 10` at line 2 is executed, function `fun()` rebinds `'x'` to a new

object whose value is 10. The two references, `'a'` and `'x'`, are uncoupled from one another. Nothing else that `fun()` does will affect `'a'`, and when `fun()` terminates, `'a'` will still point to the object 5, as it did prior to the function call. We can verify this using `id()` inside and outside of function `fun()`. See the code given below:

```
1 def fun(x):
2     print(f'id(x) = {id(x)}')
3     x = 10
4     print(f'id(x) = {id(x)}')
5
6 a = 5
7 print('Before function call...')
8 print(f'id(a) = {id(a)}')
9 fun(a)
10 print('After function call...')
11 print(f'id(a) = {id(a)}')
```

Before function call...

id(a) = 2776095981936

id(x) = 2776095981936

id(x) = 2776095982096

After function call...

id(a) = 2776095981936

The above code is a slightly modified version of the previous function and along with the value of `'a'`, we are also checked the `id`. At first, `'a'` is bind with the object 5 in line 6, and the `id(a)` before and after the function call `fun(a)` didn't change, even though we have assigned a new object to `x` inside the function `fun()` at line 3.

There is another point we need to check that, the `id(x)` at line 2 and 4 gets a different value. When we check `id(x)` at line 2, it is found that `x` inside `f()` and `a` outside of `f()`, represent the same `id`. But once we have changed the object pointed by `x` at line 3, `x's id` get modified.

We need to remember that, even though python passed arguments to a function as reference (not like C programming), once we have assigned a new object that changes will not reflect outside of the function. Argument passing in Python is somewhat of a hybrid between pass-by-value and pass-by-reference. What gets passed to the function is a reference to an object, but the reference is passed by value.

Note: Python's argument-passing mechanism has been called pass-by-assignment. This is because parameter names are bound to objects on function entries in Python, and assignment is also the process of binding a name to an object. You may also see the terms pass-by-object, pass-by-object-reference, or pass-by-sharing. The key takeaway here is that a Python function can't change the value of an argument by reassigning the corresponding parameter to something else. The following example demonstrates this:

```
1 def fun(item):
2     item = 34
3
4 for i in (2, 'Hello', [2,3,4], {'a':2, 'b':3}, {3,4}, (3,5,6)):
5     fun(i)
6     print(i)
```

2
Hello
[2, 3, 4]

```
{'a': 2, 'b': 3}
{3, 4}
(3, 5, 6)
```

Here we have passed objects of type `int`, `str`, `list`, `dict`, `set` and `tuple` to `fun()` as an argument. The function `fun()` tries to assign an `int` object `34` and once the program control comes back to the calling environment the value of the argument `i` remains unchanged. Does that mean a Python function can never modify its arguments at all? No, that isn't the case! Watch what happens here:

```
1 def fun(item):
2     if type(item) is list:
3         item.append(30) #appending an item to the list
4     elif type(item) is dict:
5         item.update({'c':9}) #updating the dictionary
6     elif type(item) is set:
7         item.add(40) #adding an item to the set
8
9 for i in ([2,3,4], {'a':2, 'b':3}, {3,4}):
10     fun(i)
11     print(i)
```

```
[2, 3, 4, 30]
{'a': 2, 'b': 3, 'c': 9}
{40, 3, 4}
```

Here we have passed mutable type objects as arguments to the function `fun()` and instead of assigning a new object to the parameter `item`, we have updated the `item` for the type `list`, `dict` and `set`. As shown in the output, we have found that once the program control returns to the calling environment changes are made inside the function that reflect outside of the function for mutable types. If the argument is a number or a string, the argument is not affected, regardless of the changes made to the parameter inside the function.

8.6 The *return* Statement

Sometimes it is necessary that the calling environment expects the calling function to return a value. What's a Python function to do then? After all, in many cases, if a function doesn't cause some change in the calling environment, then there isn't much point in calling it at all. How should a function affect its caller? Well, one possibility is to use function return values. A return statement in a Python function serves two purposes:

- It immediately terminates the function and passes execution control back to the caller.
- It provides a mechanism by which the function can pass data back to the caller.

8.6.1 Exiting from a Function

Within a function, a `return` statement causes an immediate exit from the Python function and transfers the execution back to the caller. Consider the example given below:

```
1 def fun():
2     print('start of fun()')
3     print('end of fun()')
```

```

4     return
5
6 fun()
7 print('after the fun')

```

start of fun()

end of fun()

after the fun

In this example there is no need for a **return** statement, which returns the program control back to the calling environment. Even if we didn't use the **return** statement, we are also getting the same output. It is not necessary that **return** statement should be last statement of the function, even we can place it anywhere inside the function, and even multiple times. Consider the sample code given below:

```

1 def fun(num):
2     if num > 0:
3         return
4     elif num < 0:
5         return
6     print('End of function')
7
8 fun(20)
9 fun(-1)
10 fun(0)

```

End of function

For the first two function calls at line 8 and 9, **print** statement at line 6 remain unreachable due to the pre-execution of the **return** statement at line 3, and 5. The third call **fun(0)** doesn't satisfy any of the condition of the function body written at line 2 and 4 and due to this none of the **return** statement is executed, so the **print** statement gets executed. Due to the misplaced of **return** statement sometimes it is also possible that code written inside the function body never executed. Consider the sample code given below:

```

1 def fun(num):
2     if num % 2 == 0:
3         return
4     elif num == 0:
5         return
6     else:
7         return
8     print('End of function')
9
10 fun(20)
11 fun(0)
12 fun(11)

```

The first two function call at line 10, 11 passed an argument to the function and subsequently the conditions written at line 2 and 4 become **True**. Due to this **return** statements written at line 3 and 5 get executed. Like this, third call **fun(11)** doesn't satisfy any of the conditions and the **else** clause having a **return** statement gets executed. In this example, multiple **return** statements are used and get executed before reaching the last statement at line 8 for all the function calls and the **print** statement is never executed.

Note: Once the function reached any of the **return** statement, lines written after it were never

considered for the execution. This sort of paradigm can be useful for error checking in a function. You can check several error conditions at the start of the function, with return statements that bail out if there's a problem.

```
def fun(optional_arguments):
    if error_cond1:
        return
    elif error_cond2:
        return
    elif error_cond3:
        return
    #Actual body of the function goes here
```

If none of the error conditions are encountered, then the function can proceed with its normal processing.

8.6.2 Returning values to the calling environment

The actual use of return statement is not only returning the program control to the calling environment, instead return statement is used to return some values to it. Let's see a better example to understand the use of **return**:

```
1 def fun():
2     return "I'm ok!"
3
4 msg = fun()
5 print(msg)
```

I'm ok!

The function call **fun()** at line 4 is considered as an expression and the return value from the function body **I'm ok!** is assigned to the variable **msg**. Using this mechanism, we can build communication between a calling environment and a function. In python, a function can return any type of object to the calling environment. A function call can be used syntactically anywhere in the calling environment that makes sense for the type of object the function returns.

```
1 def add(a, b): # returns a number
2     return a + b
3
4 def greater(a, b): # returns a bool
5     return True if a > b else False
6
7 def swap(a, b): # returns a list
8     a, b = b, a
9     return [a, b]
10
11 def my_dict(a, b): # returns a dictionary
12     return {'sum': a+b, 'sub': a-b, 'prod': a*b}
13
14 print(add(3, 4))
15 print(greater(10, 20))
16 swap_values = swap(3,4)
17 print(swap_values[0], swap_values[1])
18 print(my_dict(3,4))
```

The above scripts did the followings:

- Function **add()** returns the addition of two numbers of any type.
- The **greater()** function returns a Boolean type as **True** if the first parameter is larger than the second else return **False**.
- Function **swap()** interchanges the values of two parameters and returns them as a list.
- Function **my_dict()** returns a dictionary object containing the result of basic mathematical operation of two parameters.

The return type of a function also depends on how values are returned from a function or how the calling environment accepts them. Consider the example given below:

```
1 def swap(a, b): # returns multiple values
2     a, b = b, a
3     return a, b
4
5 a, b = 10, 20
6 values = swap(a, b) # received as tuple
7 print(type(values))
8 a, b = swap(a, b) # received as int object
9 print(type(a), type(b))
10 print(type(list(swap(a, b)))) # received as list
```

```
<class 'tuple'>
<class 'int'> <class 'int'>
<class 'list'>
```

The **swap()** function called thrice at line 6, 8, and 10, where function returns the values of **a** and **b** as integer after swapping. The first call at line 6, receives the return values in one variable, as tuple and this mechanism is called packing. The second call at line 8 uses two variables to hold the return values, where the number of variables is same as number of return values returns from the function. The third call received the return values as a list, as **swap()** get called inside **list()** function. In Python, it is also possible for a return statement did return nothing, instead returns the special Python value **None**, like the **pass** statement, as shown below:

```
def fun1():
    pass

def fun2():
    return

def fun3():
    print()

print(fun1(), fun2(), fun3())
```

8.6.3 Returning a function

In some scenarios it becomes compulsory to design a function whose behavior may depend on the user input that passing as argument. To handle such a scenario, we need to create functions with dynamic behavior. This can be done by nesting one function inside another, meaning outer function return an inner function. Like any other data type Python functions can be treated as first-class objects, that allows functions to be returned from other functions. The working of nesting function is summarized below:

- Define an outer function that creates an inner function. The inner function can access variables from the outer function's scope, even after the outer function has finished executing. This is because the inner function remembers the state of the outer function at the time it was created.
- The outer function returns the inner function.

Let's take one scenario where you need to create functions whose behavior depends on external factors or user input. Returning a function allows you to capture this dynamic behavior and create functions tailored to specific situations, such as unit converter.

```

1 def get_converter(unit_from):
2     """
3     This function takes a unit type and returns a function that converts
4     temperatures from that unit to Celsius.
5     """
6     def convert_to_celsius(value):
7         """
8         This function converts a temperature value to Celsius based on the unit_from.
9         """
10        if unit_from == "fahrenheit":
11            return (value - 32) * 5 / 9
12        elif unit_from == "celsius":
13            return value
14        else:
15            raise ValueError("Unsupported unit type:", unit_from)
16    return convert_to_celsius
17
18 # Get a converter function for Fahrenheit to Celsius
19 celsius_converter = get_converter("fahrenheit")
20
21 # Use the converter function
22 converted_value = celsius_converter(77)
23 print(f"{converted_value:.2f} degrees Celsius")

```

Here, `get_converter` creates a function (`convert_to_celsius`) specific to the provided unit (`fahrenheit`). This allows for code reusability and flexibility in handling different unit conversions. Explanation of the code given below:

- Line 1: Defines a function named `get_converter` that takes a single argument `unit_from`.
- Line 6: Defines another function named `convert_to_celsius` inside the `get_converter` function. This is the nested function.
- Lines 10-11: If `unit_from` is "fahrenheit", it converts from Fahrenheit to Celsius using the formula.
- Lines 12-13: If `unit_from` is "celsius", no conversion is needed, so the value is returned directly.
- Lines 14-15: If `unit_from` is not supported, it raises a `ValueError` exception.
- Line 16: The return statement in the inner function (`convert_to_celsius`) returns the calculated value or the original value depending on the unit. This return statement within the inner function is crucial. It's what makes `get_converter` a function that returns a function.
- Line 18: Calls the `get_converter` function with the argument "fahrenheit". This execution creates a new instance of the inner function (`convert_to_celsius`) specifically for converting from Fahrenheit.
- Line 19: Assigns the returned function (the conversion function for Fahrenheit to Celsius) to the variable `celsius_converter`.

- Line 21: Calls the `celsius_converter` function (which is the inner `convert_to_celsius` function) with the value 77 (presumably Fahrenheit).

In summary, the `get_converter` function acts as a factory that creates conversion functions based on the provided unit type. This allows for code reusability and avoids writing separate conversion functions for each unit.

8.7 Variable-Length Argument List

In some cases, when you're defining a function, you may not know beforehand how many arguments you'll want it to take. Suppose, for example, that you want to write a Python function that computes the average of several values. You could start with something like this:

```
1 def avg(a, b, c):
2     return (a + b + c)/3
```

This function will work fine if we call `avg()` with three arguments only. With this flavor of `avg()` passing more or lesser number of arguments will raise `TypeError` as we have seen in case of positional arguments. Further to provide some flexibility to the caller we can assign default values to the arguments and make some of them optional while calling. The sample code listed below highlights all the valid function calls from line 4 – 8.

```
1 def avg(a, b = 0, c = 0, d = 0, e = 0):
2     return (a + b + c + d + e)/5
3
4 print(avg(1))
5 print(avg(1,2))
6 print(avg(1,2,3))
7 print(avg(1,2,3,4))
8 print(avg(1,2,3,4,5))
```

```
0.2
0.6
1.2
2.0
3.0
```

But this approach still suffers from a couple of problems. For starters, it still only allows up to five arguments, not an arbitrary number. Worse yet, there's no way to distinguish between the arguments that were specified and those that were allowed to default. The function has no way to know how many arguments were passed, so it doesn't know what to divide by and always it gets divided by 5.

Without knowing the number of parameters passed from the calling environment it is difficult to return the correct average value by the function. To know the exact number of arguments, a calling environment can pass a list of items as argument and function can use `len()` to know the number of items, to produce the exact average value.

```
1 def avg(nums):
2     total = 0
3     for item in nums:
4         total += item
5     return total / len(nums)
6
7 print(avg([1]))
```

```
8 print(avg([1,2]))
9 print(avg([1,2,3]))
```

```
1.0
1.5
2.0
```

At least this works. It allows an arbitrary number of values and produces a correct result. As a bonus, it works when the argument is a tuple as well. The drawback of this approach is that the user must be aware that to group the values into a list or tuple is probably not something the user of the function would expect, and it isn't very elegant. To make the code more user-friendly python provides a better approach to pass variable number of arguments to a function using following methods:

- Argument packing as tuple using an asterisk (*) with the parameter name
- Argument packing as **key = value** pair as dictionary using double asterisk (**) with parameter name

8.7.1 Argument packing using tuple

When a parameter name in a function definition is preceded by an asterisk (*), it indicates any number of arguments are packed in a tuple. Later inside the function those parameters can be used by the given name of the parameter. Consider the example given below:

```
1 def avg(*args):
2     total = 0
3     for item in args:
4         total += item
5     return total / len(args)
6
7 print(avg([1,2,3]))
8 print(avg([1,2,3,4,5,6]))
```

In the above example function **avg()** is rewritten using the tuple packing mechanism by preceding the parameter **args** by an asterisk (*) in the function header. While receiving, parameters are packed as a single object of type tuple and inside the function the **args** considered as tuple and while calling the function, the user no need to do anything extra to group the arguments either as list or tuple as we did for the last example.

8.7.2 Arguments packing using dictionary

This process is also known as keyword argument packing. In this method we can pass variable number of arguments in a **key = value** pair, as follows:

```
1 def fun(**kwargs):
2     for key, value in kwargs.items():
3         print(f'{key}: {value}', end = ' ')
4     print()
5
6 fun(a = 2)
7 fun(a = 2, b = 3)
8 fun(a = 2, b = 3, c = 5)
```

```
a: 2
```

```
a: 2 b: 3
a: 2 b: 3 c: 5
```

The double asterisk (******) is used with function parameter name to specify the dictionary packing. In the above code **fun()** get called three times with variable arguments as **key = value** pair. Preceding a parameter name in a Python function definition by double asterisk (******) indicates that the corresponding arguments, which are expected to be **key = value** pairs, should be packed into a dictionary. While packing the arguments to a dictionary the name **kwargs** is used, you can use any other name. Here **kwargs** represents keyword argument and from the output of the code it is clear about the type of parameter **kwargs** as dictionary.

8.8 Arguments unpacking

Like tuple and dictionary packing with the parameters, reverse one means unpacking is also allowed in python. Following ways, we can unpack an argument during the function call:

- unpacking an iterable object such as tuple, list and set using an asterisk (*) with the argument name
- unpacking a dictionary as keyword argument using double asterisk (**) with argument name

8.8.1 Argument tuple unpacking

We have seen how to pass variable number of arguments by packing them to a tuple. In python we can also unpack a tuple or any iterable object before passing it as an argument to a function. Consider the example given below:

```
1 def avg(a,b,c,d):
2     return (a+b+c+d)/4
3
4 args = (3,4,5,6) #packing within a tuple
5 print(avg(*args)) #unpacking before passing the arguments
6 args = [2,3,4,5] #packing within a list
7 print(avg(*args)) #unpacking before passing the arguments
```

An asterisk (*) is placed before the name of the argument to unpack an iterable object before passing it as argument. The function call at line 5 and 7 uses an asterisk (*) before the name of argument **args** and items of the object are unpacked and then pass it to the function **avg()** as positional arguments. While unpacking any iterable it must match the number of positional parameters in the function header, else it will raise a **TypeError**.

We can also combine the packing and unpacking mechanism while passing the arguments to a function. Let's write a more user-interactive code to take input from user to find the largest of all the items. Consider the example given below:

```
1 def find_largest(*args):
2     max = args[0]
3     for i in range(1, len(args)):
4         if args[i] > max:
5             max = args[i]
6     return max
7
8 args = eval(input('Enter comma-separated values:'))
```

```
9 print(f'largest of all: {find_largest(*args)}')
```

8.8.2 Argument Dictionary Unpacking

While unpacking a dictionary the double asterisk (******) is used with argument name at the time of function calling. While unpacking a dictionary key of a dictionary should match the name of the parameters of the function. For any mismatch between a key and parameter name python will be raised a `TypeError` as got an unexpected keyword argument.

```
1 def fun(a, b, c):
2     print(f'a = {a}, b = {b}, c = {c}')
3
4 my_dict = {'a':1, 'b':2, 'c':3}
5 fun(**my_dict)
```

8.9 Keyword-only argument

As of now we have used the character ***** for two purposes (i) using ***args** to implement the variable list of arguments as a tuple and (ii) using ****kwlist** to pass only keyword list arguments as a dictionary. Suppose there is some necessity to write a function that takes any variable number of strings and concatenates them by separating an optional character. Something like this:

```
>>> def joinstr(*args):
...     print('.'.join(args))
...
>>> joinstr('a','b','c')
a.b.c
```

As of now whatever the strings we are sending it is always concatenates with a period (.) only. What if you want to modify the function to accept an argument which accepts the character also which will be used for this purpose, as the solution given below:

```
>>> def joinstr(char, *args):
...     print(char.join(args))
...
>>> joinstr('#','a','b','c')
a#b#c
```

This function could solve our purpose, but there are a couple of undesirable things about this solution:

- How could someone know that the first argument serving a specific purpose while joining other variable number of strings?
- The parameter **char** is not optional, it must be passed always.

One of the ways we can satisfy the above requirements, by making the argument **char** as default argument, which makes it optional. Let's us try this:

```
>>> def joinstr(char='.', *args):
...     print(char.join(args))
...
>>> joinstr('a','b','c')
bac
```

Unfortunately, the solution does not work, because the first string argument 'a' is considered as the special character argument, which is used for joining. Now let's us try by specifying a keyword argument before the variable number of strings as follows:

```
>>> joinstr(char='#', 'a', 'b', 'c')
File "<stdin>", line 1
    joinstr(char='#', 'a', 'b', 'c')
                                ^
```

SyntaxError: positional argument follows keyword argument

This also does not work, and the solution leads to error as positional argument follows keyword argument. So, neither of the solutions solve our purpose. Let's see how Python can help us. Syntactically the solution is very simple. The first change is to allow the optional argument to appear after a ***args** argument in function definition.

```
>>> def joinstr(*args, char='.'):
...     print(char.join(args))
...
>>> joinstr('a', 'b', 'c', char='#')
a#b#c
```

Keyword-only arguments allow a Python function to take a variable number of arguments, followed by one or more additional options as keyword arguments. If you wanted to modify **joinstr()** so that a separator (**sep**) along with an **ends** character can optionally be specified as well, by adding an additional keyword-only argument:

```
>>> def joinstr(*args, sep = ' ', ends = '.'):
...     print(sep.join(args) + ends)
...
>>> joinstr('This', 'is', 'Python', '3.10', sep = '|', ends = '!')
This|is|Python|3.10!
```

The following points need to remember to work with keyword-only arguments: (a) If the keyword argument didn't pass at the time of calling then the default value will be considered. (b) If the default value didn't provide then passing the values for the keyword argument is compulsory. Not doing so will raise an error.

The second approach to implement the keyword-only argument is achieved by a syntactical change, which allow the argument name to be omitted for a variable list argument (varargs), as follows:

```
>>> def fun(a, b, *, karg = None):
...     pass
```

This approach is useful when we want to define a function is not allowed to take variable list arguments and take a keyword argument. Consider the example given below, which performs a designated operation over two arguments:

```
1 def calculator(a, b, ope='+'):
2     if ope == '+':
3         return a + b
4     elif ope == '-':
5         return a - b
6     elif ope == '*':
7         return a * b
```

```

8     elif ope == '/':
9         return a - b
10    else:
11        return None

```

For the above function the type of operation performed it depends on the **ope** parameter. If we want the **ope** argument should pass as keyword only, we can add an unnamed ***** parameter before the **ope**. In this case the **ope** argument become compulsory to pass as keyword-only or optional, because of the default value assigned to it. Now check the next three function calls:

```

>>> def calculator(a, b, *, ope='+'):
...     if ope == '+':
...         return a + b
...     elif ope == '-':
...         return a - b
...     elif ope == '*':
...         return a * b
...     elif ope == '/':
...         return a - b
...     else:
...         return None
...
>>> calculator(3,4, ope = '-')
-1
>>> calculator(3,4)
7
>>> calculator(3,4,'-')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: calculator() takes 2 positional arguments but 3 were given

```

First two calls work fine, but the third call pass the **ope** argument as positional which leads to an exception. Because it considered it as positional argument. What if you pass some extraneous positional argument with two required positional arguments and keyword-only argument, as follows:

```

>>> calculator(3,4,"no extra values",'-')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: calculator() takes 2 positional arguments but 4 were given

```

The **calculate()** function is not ready yet to take any extra positional argument that can be ignored by the function. For this we can make a dummy argument by using a name with the ***** and later ignored by the function this as follows:

```

>>> def calculator(a, b, *ignore, ope='+'):
...     print(ignore)
...     if ope == '+':
...         return a + b
...     elif ope == '-':
...         return a - b
...     elif ope == '*':
...         return a * b
...     elif ope == '/':
...         return a - b
...     else:

```

```

...     return None
...
...
>>> calculator(3,4,"no extra values",ope = '/')
('no extra values',)
-1

```

It takes only two positional arguments and any extra positional argument pass by the caller will be ignored and the respective operation is performed depends on the **ope** passed as keyword-only argument.

8.10 Positional-only Argument

Python 3.8 introduces positional-only arguments as compare with keyword-only argument. Means the corresponding arguments must be supplied positionally and can't be specified by keyword. Using a forward-slash (/) we can make some parameters as positional-only.

Any parameters to the left of the slash (/) must be specified positionally. For example, in the following function definition, **a** and **b** are positional-only parameters, but **c** may be specified by keyword:

```

>>> def fun(a, b, /, c):
...     print(a, b, c)
...
>>> fun(2, 3, c = 4) #c as keyword
2 3 4
>>> fun(2, 3, 4) #c as positional
2 3 4

```

Above two function calls don't test the actual meaning of **fun()**, which accepts **a** and **b** as positional only. Let's try to send any of these two parameters as keyword and trying to do so leads to a **TypeError** as **fun()** expecting **a** and **b** as positional but pass as keyword.

```

>>> fun(a = 2, b = 3, c = 4)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: fun() got some positional-only arguments passed as keyword arguments: 'a, b'

```

The positional-only arguments can be mixed with keyword-only arguments also, if it is necessary as follows:

```

>>> def fun(a, b, /, *, c):
...     print(a, b, c)
...
>>> fun(2, 3, c = 4) #c must be pass keyword only
2 3 4

```

In the above function definition, an unnamed argument ***** is used after the forward slash (/) and it makes anything passed after the positional argument **b** should be passed as keyword only and not doing so will raised a **TypeError**.

8.11 The docstring

A docstring is a string literal that occurs as the first statement in a module, function, class, or method definition. Such a docstring becomes the **__doc__** special attribute of that object. String literals occurring elsewhere in Python code may also act as documentation and can be retrieve using a software

tool. The following two string literal types can't be retrieved using the runtime object `__doc__`.

- String literals occurring immediately after a simple assignment at the top level of a module, class, or `__init__` method is called “attribute docstrings”.
- String literals occurring immediately after another docstring are called “additional docstrings”.

This section highlights docstrings related to a function and docstring related to class and module is discussed in vol. 2 of this book. In python following ways we can prepare the docstrings:

- For consistency, always use `"""triple double quotes"""` around docstrings.
- To represent a backslash as part of the docstring use `r"""raw triple double quotes"""`.
- For Unicode docstrings, use `u"""Unicode triple-quoted strings"""`.

Note: A docstring is used to supply documentation for a function. It can contain the function's purpose, what arguments it takes, information about return values, or any other information you think would be useful.

8.11.1 One-line docstring

A one-liner docstring should really fit on one line, as follows:

```
>>> def function(arg):
...     """This is __doc__, function's docstring"""
...     pass
... 
```

Following list of points need to be taken care while writing a one-liner docstring:

- Triple quotes are used even though the string fits on one line. This makes it easy to later expand it as multi-liner docstring
- The closing quotes are on the same line as the opening quotes. This looks better for one-liners.
- There's no blank line either before or after the docstring.
- The docstring is a phrase ending in a period. It prescribes the function or method's effect as a command (“Do this”, “Return that”), not as a description.

```
>>> def fun2(a,b):
...     """perform addition of a and b and return the result"""
...     pass
... 
```

```
>>> print(fun2.__doc__)
perform addition of a and b and return the result
```

The runtime object `__doc__` is used to retrieve the docstring of a function, as shown in the output. The one-line docstring should NOT be a “signature” of a function reiterating the function/method parameters (which can be obtained by introspection). Don't do:

```
>>> def fun1(a, b):
...     """fun1(a,b) -> list"""
...     pass
... 
```

The `help()` function is one of the ways to do code introspection, as follows:

```
>>> help(fun1)
Help on function fun1 in module __main__:

fun1(a, b)
    fun1(a,b) -> list
```

8.11.2 Multi-line docstring

Multi-line docstrings consist of a summary line just like a one-line docstring, followed by a blank line, followed by a more elaborate description. An example such type is given below:

```
def getcomplex(real=0.0, imag=0.0):
    """Form a complex number.

    Keyword arguments:
        real: the real part (default 0.0)
        imag: the imaginary part (default 0.0)

    Returns:
        A complex number.
    """

    if imag == 0.0 and real == 0.0:
        return 0j
    elif imag == 0.0:
        return real + 0j
    else:
        return complex(real, imag)

# Print the docstring
print(getcomplex.__doc__)

# Print the result of calling getcomplex() with no arguments
print(getcomplex())

# Print the result of calling getcomplex() with real=3
print(getcomplex(3))

# Print the result of calling getcomplex() with real=4 and imag=5
print(getcomplex(4, 5))
```

Like single line docstring multi-liner is also retrieve by either `help()` function or special attribute `__doc__` as shown above. Further details about the docstring indentation, docstring inside a class or module read the documents provided as footnote^{23,24}.

²³ D. G. a. G. v. Rossum, "PEP 257 – Docstring Conventions," 29 5 2001. [Online]. Available: <https://peps.python.org/pep-0257/>.

²⁴ D. Goodger, "PEP 258 – Docutils Design Specification," 31 5 2001. [Online]. Available: <https://peps.python.org/pep-0258/#attribute-docstrings> . [Accessed 22 10 2022]

8.12 Topics Not Covered

To keep the discussion limited about python function and its features following list of topics are sifted to vol. 2 of this book.

- (a) lambda or anonymous function
- (b) function recursion
- (c) function annotations
- (d) Scope rules

*****END OF CHAPTER*****